



Quiz de Fixação — Os Arquivos do AlunosApp

15 questões objetivas sobre o papel de cada arquivo do projeto. Responda com calma, clique em **Verificar Respostas** no final e veja o gabarito comentado de cada questão.

Respondidas:

15 / 15

1 Qual arquivo é responsável por montar a **estrutura visual** da tela (título, campos, botões e tabela)?

☐ app.js

☒ index.html

☐ style.css

☐ manifest.json

✓ **Correto!** O `index.html` é o esqueleto da tela. Ele contém os campos, o botão e a tabela — mas sem estilo (CSS) e sem lógica (JS).

2 Se o arquivo `app.js` estiver com erro ou faltando, o que acontece?

☐ O app abre em branco, sem nenhum elemento visível.

☒ Os botões não funcionam; a tabela pode exibir os textos literais `${a.id}`.

☐ O ícone do app desaparece da tela inicial.

☐ O GitHub Pages deixa de publicar o site.

✓ **Correto!** Sem o `app.js`, ninguém escuta os cliques nem atualiza a tabela. O sintoma clássico é a tabela exibindo `${a.id}` literalmente.

3 Para que serve o arquivo `style.css` ?

☐ Guardar os dados no LocalStorage.

☒ **Definir a aparência: cores, fontes, espaçamentos, bordas.**

☐ Permitir a instalação como PWA.

☐ Compilar o APK do Android.

✓ **Correto!** O `style.css` cuida só da aparência. Sem ele, o app funciona, mas fica visualmente "cru", parecido com um site antigo.

4 O arquivo `manifest.json` é a "carteira de identidade" do PWA. O que ele contém?

☐ A lógica do CRUD em JavaScript.

☒ **Nome do app, nome curto, cor do tema e lista de ícones.**

☐ Os comandos do Capacitor para gerar o APK.

☐ A folha de estilos escura do app.

✓ **Correto!** O `manifest.json` traz nome, nome curto, cores, ícones e o `display: standalone`, que faz o app abrir em tela cheia.

5 Se o `manifest.json` não existir, o que o usuário deixa de ver no Chrome Android?

☐ A tabela de alunos cadastrados.

☒ **O aviso "Adicionar à tela inicial".**

☐ O botão "Salvar Aluno".

☐ O título da página.

✓ **Correto!** Sem o `manifest.json`, o Chrome não reconhece o site como PWA instalável e não oferece "Adicionar à tela inicial".

6 Qual arquivo faz o app funcionar **offline**, guardando cópia dos arquivos no cache do celular?

☐ `index.html`

☒ **`sw.js` (Service Worker)**

☐ `build.yml`

☐ `manifest.json`

✓ **Correto!** O `sw.js` (Service Worker) guarda cópias dos arquivos no cache local e responde requisições mesmo sem internet.

7 Se o arquivo `sw.js` não existir, qual é a consequência mais visível?

☐ O app fica feio, sem cores.

☒ **O app não abre offline e o Chrome para de oferecer instalação.**

☐ O botão Excluir não funciona.

☐ A tabela deixa de mostrar as idades.

✓ **Correto!** Sem o `sw.js`, o app não abre offline, o Chrome para de oferecer instalação e o site pode ficar preso em cache antigo.

8 Sobre os arquivos de ícone `icon-192.png` e `icon-512.png`, é correto afirmar que:

☐ Podem ser substituídos por arquivos `.txt`, pois o navegador ignora o formato.

☒ **Precisam ser PNG reais e existir nos tamanhos exatos que o `manifest.json` declara.**

☐ Devem ficar dentro da pasta `.github/workflows/`.

☐ Substituem o papel do `index.html` como página inicial.

☒ **Correto!** Os ícones precisam ser PNG reais nos tamanhos 192×192 e 512×512 — os mesmos declarados no `manifest.json`. Sem eles, o PWA não instala.

9 O que o arquivo `build.yml` faz?

☐ Guarda os dados dos alunos no celular.

☐ Define as cores e o layout do app.

☒ **Dá instruções ao GitHub Actions para publicar o site e gerar o APK automaticamente.**

☐ Registra o Service Worker no navegador.

☒ **Correto!** O `build.yml` é a "receita" que o GitHub Actions executa: publica o site no Pages e gera o APK com Capacitor.

10 Em qual caminho o `build.yml` deve estar para o GitHub reconhecê-lo?

☐ Na raiz do repositório, junto com o README.

☐ Dentro de `www/`, ao lado do `app.js`.

☒ **Em `.github/workflows/build.yml`, exatamente com esse caminho.**

☐ Em qualquer pasta, desde que o nome do arquivo termine em `.yml`.

☒ **Correto!** O caminho exato é `.github/workflows/build.yml`. Qualquer variação (sem o ponto, sem o "s", com tab) faz o GitHub ignorar o workflow.

11 Por que o `build.yml` não pode ter tabulações (Tab), apenas espaços?

☐ Porque tabs ocupam muito espaço no arquivo.

☒ **Porque o formato YAML usa indentação como sintaxe e não aceita Tab — o GitHub ignora o workflow silenciosamente.**

☐ Porque o Windows transforma tabs em caracteres especiais.

☐ Porque o Java não consegue ler tabs.

✓ **Correto!** YAML usa indentação como sintaxe. Tabs quebram a leitura e o arquivo é ignorado silenciosamente — sem aviso de erro.

12 **Ao editar qualquer arquivo do app, por que é importante trocar `alunosapp-v1` para `alunosapp-v2` dentro do `sw.js` ?**

☐ Para o app ficar mais rápido.

☒ **Para o Service Worker descartar o cache antigo e baixar a versão nova — senão o celular continua mostrando a versão velha.**

☐ Para o APK ficar menor.

☐ Porque o GitHub não aceita dois commits com o mesmo texto.

✓ **Correto!** A versão no `sw.js` força o Service Worker a descartar o cache antigo e buscar os arquivos novos. Sem isso, o celular continua com a versão antiga.

13 **O mesmo arquivo `index.html` é usado tanto para o site publicado no GitHub Pages quanto para o app dentro do APK Android.**

☒ **Verdadeiro**

☐ **Falso**

✓ **Correto!** O mesmo código HTML é publicado no Pages como PWA e empacotado no APK pelo Capacitor. Isso é o que permite manter uma única base de código.

14 **O arquivo `app.js` é executado tanto no navegador (PWA) quanto dentro do WebView do APK, pois ambos têm motor JavaScript.**

☒ **Verdadeiro**☐ **Falso**

✓ **Correto!** Tanto o navegador quanto o WebView do Android têm um motor JavaScript. Por isso o `app.js` funciona idêntico nos dois ambientes.

15 Se um único arquivo dos 7 (por exemplo, o `sw.js`) estiver faltando, o app pode até abrir, mas ficará com alguma funcionalidade comprometida.

☒ **Verdadeiro**☐ **Falso**

✓ **Correto!** Cada arquivo cobre uma parte específica. Se algum faltar, o app até pode abrir, mas perde uma funcionalidade — offline, instalação, aparência ou lógica.

☒ Verificar Respostas Limpar Respostas

Resultado Final

15/15

Aproveitamento: 100%



Excelente! Você domina o papel de cada arquivo do projeto. Está pronto para aplicar em novos apps.